

MessageFormat 2 — Architecture

- Author: Alexey Lyakhov

Companion to `messageformat-v2-research.md` and `messageformat-v2-implementation-details.md`.

This document defines the crate layout, module split, type names, error shape, and integration points. It is the **architecture contract** the phased commit plan (`-implementation-details.md`) must honor.

1. Crate placement decision

Start as a sub-module of `icu_experimental` , not a new top-level crate.

- Follows the precedent of `relativetime` , `displaynames` , `personnames` , `transliterate` , `units` , `dimension` , `duration` , `measure` — all pre-1.0 components live inside `components/experimental/src/*` .
- Gets the `unstable` / `compiled_data` / `serde` / `datagen` feature matrix for free from the host crate.
- Avoids a workspace-level `Cargo.toml` change and an FFI surface on day one (FFI is deferred until the API settles — `documents/process/graduation.md`).

Graduation path (post-stabilization):

1. Move `components/experimental/src/messageformat/` → `components/messageformat/` .
2. Create matching `provider/data/messageformat/` baked-data crate.
3. Add FFI bindings in `ffi/capi/src/messageformat.rs` .
4. Add to `components/icu/` re-exports.

Crate module path while experimental: `icu::experimental::messageformat` .

Graduated path: `icu::messageformat` (Rust) / `icu_messageformat` (direct).

2. Directory layout

```
components/experimental/src/messageformat/
├─ mod.rs                # public re-exports, crate doc
├─ ast.rs                # Data model (§4). Owned, `serde`/`databake` optional
├─ parser.rs             # MF2 source -> ast::Message
├─ parser/
│   ├─ lexer.rs          # Character-level tokenizer
│   └─ grammar.rs        # Recursive-descent over tokens
├─ validator.rs          # Data-model validation pass
├─ resolver.rs           # Variable + declaration resolution
├─ selector.rs           # Pattern-selection algorithm (§6)
├─ formatter.rs          # MessageFormatter + builder (§5)
├─ function/
│   ├─ mod.rs            # Function / Selector traits (§7)
│   ├─ registry.rs       # FunctionRegistry
│   └─ builtin/
│       ├─ string.rs     # `:string`
│       ├─ number.rs     # `:number` / `:integer`
│       ├─ datetime.rs   # `:date` / `:time` / `:datetime` (unstable)
│       └─ currency.rs   # stretch: `:currency` / `:percent` / `:unit`
│   └─ u_namespace.rs    # `u:id`, `u:dir`, `u:locale`
├─ value.rs              # ResolvedValue + MessageValue
├─ bidi.rs               # Directionality detection + isolates
├─ parts.rs              # FormattedPart / PartsWriter
├─ error.rs              # Error enums (§8)
└─ provider.rs           # DataMarkers (§10)

components/experimental/tests/messageformat/
├─ tests.rs              # top-level integration entry (driven from Cargo.toml)
├─ conformance.rs        # runs the vendored WG JSON test suite
└─ fixtures/             # vendored snapshot of message-format-wg/test/**
    └─ README.md          # pinned commit + sync instructions
```

A `[[test]] messageformat_test` entry in `components/experimental/Cargo.toml` mirrors the existing `displaynames_test`, `relativetime_test` style.

3. Public API sketch

```
use icu::experimental::messageformat::{
    Message, MessageFormatter, MessageFormatterBuilder, FormatError, FormatOptions,
};
use icu::locale::locale;

let formatter = MessageFormatter::builder()
    .locale(locale!("en-GB"))
    .source(".input {$count :integer}\\n.match $count\\n\\
        0    {{You have no notifications.}}\\n\\
        one {{You have {$count} notification.}}\\n\\
        *    {{You have {$count} notifications.}}")?
    .build()?;

let mut out = String::new();
let errors = formatter
    .format(&inputs, &mut out)?;    // Writeable sink; returns Vec<FormatError>
```

Three canonical entry points, mirroring the JS reference:

```
// Parse once, format many.
let ast: Message = Message::parse(src)?;                // => Result<_, ParseError>
let formatter = MessageFormatter::builder()
    .locale(loc)
    .message(ast)                                       // accept either source or pre-parsed
    .build()?;

// Format to a Writeable.
let mut buf = String::new();
let errs = formatter.format(&inputs, &mut buf)?;

// Format to structured parts (feature = "parts" or always-on).
let parts: Vec<FormattedPart> = formatter.format_to_parts(&inputs)?;
```

Important: `format` returns `Result<Vec<FormatError>, core::fmt::Error>`.

The `Ok` variant carries the list of emitted resolution / function errors

(empty on success). The `Err` variant is reserved for Writeable I/O

failure — never for MF2 semantics. This matches the spec requirement that

every error must be surfaced and matches the JS library's `onError` contract.

4. Data model (ast.rs)

```
// Owned model. `Cow<'a, str>` allows zero-copy parsing when `alloc` is on.
#[non_exhaustive]
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Message {
    Pattern { declarations: Vec<Declaration>, pattern: Pattern },
    Select { declarations: Vec<Declaration>, selectors: Vec<VariableRef>, variants: Vec<Variation> }
}

#[non_exhaustive]
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Declaration {
    Input { name: Name, value: VariableExpression },
    Local { name: Name, value: Expression },
    // Reserved-statement forms (future spec extensions) live here.
}

pub type Pattern = Vec<PatternElement>;

#[non_exhaustive]
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum PatternElement {
    Text(String),
    Expression(Expression),
    Markup(Markup),
}

#[non_exhaustive]
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Expression {
    Literal(LiteralExpression),
    Variable(VariableExpression),
    Function(FunctionExpression),
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct LiteralExpression { pub arg: Literal, pub function: Option<FunctionRef>, pub attributes: Attributes }
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct VariableExpression { pub arg: VariableRef, pub function: Option<FunctionRef>, pub attributes: Attributes }
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct FunctionExpression { pub function: FunctionRef, pub attributes: Attributes }

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct FunctionRef { pub name: Identifier, pub options: OptionMap }
```

```

pub type OptionMap = Vec<(Identifier, OptionValue)>; // preserves insertion order; dup-check ru
#[non_exhaustive]
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum OptionValue { Literal(Literal), Variable(VariableRef) }

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Markup { pub kind: MarkupKind, pub name: Identifier, pub options: OptionMap, pub att
#[non_exhaustive]
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum MarkupKind { Open, Standalone, Close }

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Variant { pub keys: Vec<VariantKey>, pub value: Pattern }
#[non_exhaustive]
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum VariantKey { Literal(Literal), Catchall { value: Option<Name> } }

pub type Attributes = Vec<(Identifier, AttributeValue)>;
#[non_exhaustive]
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum AttributeValue { Literal(Literal), Present }

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Identifier { pub namespace: Option<Name>, pub name: Name }

pub type Name = Box<str>;          // NFC-normalised on construction

#[non_exhaustive]
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Literal { Unquoted(Box<str>), Quoted(Box<str>) }

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct VariableRef { pub name: Name }

```

Conformance rules:

- Every `pub enum` carries `#[non_exhaustive]` (workspace lint `clippy::exhaustive_enums = "deny"` forces this).
- Every `pub struct` either carries `#[non_exhaustive]` or has fields that will never need additions (only the trivial refs here). Review during Phase 1.
- `Debug` is derived everywhere; workspace has `rust.missing_debug_implementations = "deny"`.
- `serde::Serialize` / `serde::Deserialize` are provided behind `feature = "serde"` for the whole AST so that parsed messages can be shipped as JSON (matching the spec's JSON Schema interchange format).

- `data bake::Bake` is provided behind `feature = "datagen"` so that parsed messages can be baked into data.

5. Formatter

```
pub struct MessageFormatter { /* private */ }

impl MessageFormatter {
    pub fn builder() -> MessageFormatterBuilder { .. }

    pub fn format<V, W>(&self, values: &V, out: &mut W) -> Result<Vec<FormatError>, core::fmt::Write>
    where V: InputValues, W: Writeable + fmt::Write;

    pub fn format_to_string<V>(&self, values: &V) -> (String, Vec<FormatError>)
    where V: InputValues;

    pub fn format_to_parts<V>(&self, values: &V) -> (Vec<FormattedPart>, Vec<FormatError>)
    where V: InputValues;
}

pub struct MessageFormatterBuilder { /* private */ }
impl MessageFormatterBuilder {
    pub fn locale(self, loc: Locale) -> Self;
    pub fn locale_preferences(self, prefs: LocalePreferences) -> Self;
    pub fn bidi_isolation(self, on: bool) -> Self;
    pub fn source(self, src: &str) -> Result<Self, ParseError>;
    pub fn message(self, msg: Message) -> Self;
    pub fn function<F: FunctionHandler + 'static>(self, id: Identifier, f: F) -> Self;
    pub fn functions<I>(self, reg: FunctionRegistry) -> Self;

    // Advanced: swap the data provider (datagen / dynamic data)
    #[cfg(feature = "unstable")]
    pub fn unstable_with_provider<P: DataProvider<..>>(self, provider: P) -> Self;

    pub fn build(self) -> Result<MessageFormatter, BuildError>;
}
```

InputValues trait

```
pub trait InputValues {  
    fn get(&self, name: &str) -> Option<InputValue<'_>>;  
}  
  
pub enum InputValue<'a> {  
    Null,  
    Bool(bool),  
    Number(Decimal),           // fixed_decimal::Decimal  
    String(&'a str),  
    DateTime(IsoDateTime),     // icu_time::IsoDateTime  
    Opaque(&'a dyn AnyValue),  // trait-object escape hatch  
}
```

Implementors: `BTreeMap<String, InputValue>`, `litemap::LiteMap`, and a derive macro could be added later. Start with a manual impl for `&[(&str, InputValue)]` and `BTreeMap<_, _>`.

6. Selection algorithm

Implemented in `selector.rs`, per spec §5 *Pattern Selection*:

1. Resolve every selector expression, producing a `ResolvedValue` each.
2. For every variant `v`, compute `matches[i] = selector[i].match(key[i])` where `key[i]` is `v.keys[i]`. Catchall keys always match.
3. Drop variants where any `matches[i]` is `NoMatch`.
4. Sort the survivors by the selectors' `better_than` relation, lexicographically by selector index (stable sort, highest preference first).
5. Return the first variant. If none matched, return the all-catchall variant (guaranteed to exist by the validator; else the formatter would not build).
6. Any selector whose resolution failed contributes a *Bad Selector* error and forces the all-catchall fallback.

7. Function handlers

```
pub trait FunctionHandler: Send + Sync {
    /// Resolve: produce a ResolvedValue from operand + options.
    fn resolve<'ctx>(
        &self,
        ctx: &FunctionContext<'ctx>,
        operand: Option<&ResolvedValue>,
        options: &ResolvedOptions,
    ) -> Result<ResolvedValue, FunctionError>;
}

pub struct FunctionContext<'a> {
    pub locale:      &'a LocalePreferences,
    pub direction:   Direction,           // from bidi
    pub data_provider: &'a dyn AnyProvider,
}

#[non_exhaustive]
pub struct ResolvedValue {
    pub part_kind: &'static str,          // "number", "string", "datetime", ...
    pub value:     Box<dyn AnyValue>,      // typed inner value
    pub format:    Box<dyn Formattable>,   // knows how to write itself
    pub selector:  Option<Box<dyn SelectorImpl>>,
    pub dir:       Option<Direction>,      // honours u:dir override
    pub id:        Option<Box<str>>,        // honours u:id
}

pub trait Formattable {
    fn write(&self, w: &mut dyn fmt::Write) -> fmt::Result;
    fn to_part(&self) -> FormattedPart;
}

pub trait SelectorImpl {
    fn matches(&self, key: &Literal) -> KeyMatch;           // Exact, PluralKeyword, NoMa
    fn better_than(&self, a: &VariantKey, b: &VariantKey) -> Ordering;
}
```

A `FunctionRegistry` is a `BTreeMap<Identifier, Arc<dyn FunctionHandler>>` with $O(\log n)$ lookup, stable iteration for diagnostics, and efficient clone via `Arc`. Default registry is assembled by `FunctionRegistry::default_registry()`; individual features ("mf2-datetime-functions") gate draft entries.

8. Errors

```
#[non_exhaustive]
#[derive(Debug, displaydoc::Display)]
pub enum ParseError {
    /// Syntax error at byte {offset}: {message}
    Syntax { offset: usize, message: &'static str },
    /// Unexpected end of input
    UnexpectedEof,
    // ...
}
```

```
#[non_exhaustive]
#[derive(Debug, displaydoc::Display)]
pub enum ValidationError {
    /// Missing fallback variant
    MissingFallbackVariant,
    /// Variant key count {actual} does not match selector count {expected}
    VariantKeyMismatch { expected: usize, actual: usize },
    /// Duplicate declaration of ${name}
    DuplicateDeclaration { name: Box<str> },
    /// Duplicate variant
    DuplicateVariant,
    /// Duplicate option `${name}`
    DuplicateOptionName { name: Box<str> },
    /// Missing selector annotation on ${name}
    MissingSelectorAnnotation { name: Box<str> },
}
```

```
#[non_exhaustive]
#[derive(Debug, displaydoc::Display)]
pub enum BuildError {
    /// Parse error: {0}
    Parse(ParseError),
    /// Validation error: {0}
    Validation(ValidationError),
    /// Data provider error: {0}
    Data(DataError),
}
```

```
#[non_exhaustive]
#[derive(Debug, Clone, displaydoc::Display)]
pub enum FormatError {
    /// Unresolved variable: ${name}
    UnresolvedVariable { name: Box<str> },
    /// Unknown function: :{name}
```

```

UnknownFunction      { name: Box<str> },
/// Bad selector for ${name}
BadSelector          { name: Box<str> },
/// Function error: {error}
FunctionError        { function: Box<str>, error: FunctionError },
}

#[non_exhaustive]
#[derive(Debug, Clone, displaydoc::Display)]
pub enum FunctionError {
    /// Bad operand
    BadOperand,
    /// Bad option `{name}`
    BadOption { name: Box<str> },
    /// Unsupported operation
    UnsupportedOperation,
    /// Other: {message}
    Other { message: Box<str> },
}

```

All use `displaydoc` for `Display` (no direct `thiserror` dep; matches the rest of `icu4x`).

9. Bidi

- Detected directionality of the base message: locale-level (`icu_locale` character direction) unless overridden at build time.
- Placeholder direction: inherit by default; `u:dir=ltr|rtl|auto` overrides.
- `auto` uses first-strong-character detection from `icu_properties` (`Bidi_Class`).
- Isolation in concatenated string output: LRI/RLI/FSI + PDI around the rendered placeholder, only when the placeholder's direction differs from the base AND `bidi_isolation` is enabled (default).
- Structured parts output never applies isolates (the consumer handles it).

10. Data markers

Minimal set for the v1 stable surface (`:string` + `:integer` + `:number`):

Marker	Backed by	Used for
existing PluralsCardinalV1	icu_plurals	:number select=plural
existing PluralsOrdinalV1	icu_plurals	:number select=ordinal
existing DecimalSymbolsV1	icu_decimal	:number / :integer formatting
existing CharacterDirectionV1	icu_properties	bidi first-strong

Draft-function markers (`:date` , `:time` , `:datetime`) come from `icu_datetime` . No new markers from MF2 itself for the v1 scope — this is deliberate, keeps `provider/data/experimental` untouched.

If a custom function namespace ships (e.g. CLDR-derived grammatical gender tables for a future `:person`), it will introduce its own markers in a follow-up; not in scope here.

11. no_std, alloc, features

The crate is `no_std` + `alloc` . Features on the *experimental* host crate:

Feature	Effect
serde	Enables <code>serde::{Serialize,Deserialize}</code> on <code>ast::*</code> , error types.
datagen	Enables <code>atabake::Bake</code> and baked data export.
compiled_data	Enables the default baked data for <code>:number / :string</code> .
unstable	Gates draft-function handlers (<code>:date / :time / :datetime</code>), the <code>unstable_with_provider</code> builder method, the raw data-model types.
log	Debug logging.

No `std` -only APIs. All I/O goes through `Writeable / core::fmt::Write` .

12. Integration points with existing icu4x crates

- `icu_locale_core::LocalePreferences` — locale negotiation input to every function handler.
- `icu_plurals::PluralRules` — `:number / :integer` selector.

- `icu_decimal::DecimalFormatter` — `:number / :integer` formatter.
- `fixed_decimal::Decimal` — numeric operand representation.
- `icu_datetime::DateTimeFormatter` — draft `:date / :time / :datetime`.
- `icu_properties (Bidi_Class)` — bidi detection in `:string auto`.
- `icu_normalizer::ComposingNormalizer` — NFC comparison in `:string` selection and in identifier equality.
- `writeable::Writeable` — output trait implemented by `FormattedMessage<'_>`.
- `icu_pattern::SinglePlaceholderPattern` — considered, not used: MF2's shape is richer. Reuse its `Writeable` idioms, not its parser.

13. Conformance-test harness

A new test binary `messageformat_test` (driven by the `tests/messageformat/` tree referenced in `components/experimental/Cargo.toml`).

Implementation:

1. `tests/messageformat/fixtures/` is a **vendored** snapshot of `message-format-wg/test/`, pinned by commit SHA in a sibling `fixtures/README.md`. A makefile target `cargo make sync-mf2-tests` re-syncs it.
2. `tests/messageformat/conformance.rs` walks `fixtures/tests/**/*.json`, deserializes each test case via `serde_json`, and invokes the formatter. For each case it asserts:
 - `exp` matches the formatted string (when present).
 - `expErrors` set-equals the returned `FormatError` tags (order-insensitive; the spec does not constrain order).
 - `expParts` matches `format_to_parts` output (when present).
3. The `:test:select / :test:format` harness functions mandated by the spec's appendix are registered by the test module via a `register_conformance_functions(&mut reg)` helper that does **not** ship in the public API.

A failing test prints a compact diff so that conformance regressions are immediately actionable.

14. Lints and code style contract

Every file begins with the canonical license header:

```
// This file is part of ICU4X. For terms of use, please see the file
// called LICENSE at the top level of the ICU4X source tree
// (online at: https://github.com/unicode-org/icu4x/blob/main/LICENSE ).
```

`mod.rs` of the submodule gets:

```
// Relax root-level lint denials that are applied to icu_experimental/src/lib.rs
// only for the duration of experimental work. These are re-enabled on graduation.
#![allow(clippy::panic, clippy::indexing_slicing, clippy::unwrap_used, missing_docs)]
```

(matches the existing submodules; drop this allow-block when the crate graduates to `components/messageformat/`.)

All enums: `#[non_exhaustive]` . All pub structs: explicit decision per type.

All pub items: `Debug` . All MSRV-compatible with the workspace (currently 1.86, on the toolchain-1.95 recent CI).

15. Design tradeoffs & deferred items

Decision	Choice	Alternative rejected	Why
Crate placement	<code>icu_experimental</code> submodule	New top-level crate	Matches precedent; avoids FFI/graduation costs
Parsing strategy	Hand-written recursive descent	<code>nom</code> / <code>winnow</code> / <code>combine</code>	Zero extra deps, full control of error offsets, <code>no_std</code> clean
Error reporting	<code>Result<Vec<FormatError>, fmt::Error></code>	Return struct with both	Single return type keeps callers simple
Lazy evaluation	Call-by-need via an internal resolution cache	Eager resolution	Spec mandates <i>at most once</i> ;

Decision	Choice	Alternative rejected	Why
			handlers may have side effects
Function registry	<code>Arc<dyn FunctionHandler> map</code>	Enum of built-ins	Must allow user extension
Draft functions	<code>feature = "unstable"</code>	Always on	Spec marks them Draft; we respect the gate
Parts output	Always available	Feature-gated	Spec requires both; parts are cheap
Data model owned	Owned strings/ <code>Box<str></code>	Zero-copy via Cow	Simpler v1; revisit once benchmarks land
Input values	Enum + opaque trait object	Generic over <code>T: ToInputValue</code>	Monomorphization blow-up not worth it for typical message sizes

Explicit non-goals:

- Fluent (`.ftl`) importer — future `icu_fluent` or a feature of this crate.
- MF1 importer — future `icu_messageformat_compat` crate.
- XLIFF 2 serializer — tooling layer, out of scope.
- FFI — deferred until graduation.
- No-alloc mode — not pursued; MF2 fundamentally requires dynamic lookups.